

MULTI-CLOUD STORAGE COMPATIBILITY

Connect Your Existing Storage Tools to Inaya

A step-by-step guide for pilot IT/ops teams: use your existing AWS CLI, Azure tooling, and SDKs against Inaya's real S3- and Azure-compatible endpoints — no new tools to install, no workflow changes.

SECTION 01

What This Gives You

Inaya exposes two real, standards-based storage endpoints. Anything that already speaks AWS S3 or Azure Blob Storage — the AWS CLI, boto3/the AWS SDKs, rclone, Terraform's S3 backend, Cyberduck, the Azure CLI, the Azure Storage SDKs — can point at Inaya directly, using its own existing configuration options for a custom endpoint. Every byte you send is still encrypted, sharded, and redundantly stored across Inaya's decentralized storage network underneath — the compatibility layer is a real protocol translation, not a re-upload to someone else's cloud.

PROTOCOL	ENDPOINT	WHAT IT'S FOR
AWS S3-compatible	https://<your-inaya-host>/api/s3	Any AWS CLI/SDK/tool that supports a custom S3 endpoint
Azure Blob-compatible	https://<your-inaya-host>/api/azure	Any Azure CLI/SDK/tool that supports a custom Blob Storage endpoint

What to put in place of <your-inaya-host>. For the standard, shared Inaya Network platform, replace it with www.inayanetwork.com — so the two endpoints above become <https://www.inayanetwork.com/api/s3> and <https://www.inayanetwork.com/api/azure>. Every command example later in this guide uses the same placeholder; substitute the same value everywhere it appears. If your pilot is running on a dedicated or private instance instead of the shared platform, your Inaya account manager will give you that instance's specific hostname to use here instead.

SECTION 02

Step 1 — Get Your Credentials

1. **Sign in to Business Workspace** Go to your organization's Business Workspace and open the storage section from the left navigation.
2. **Open "S3-Compatible Storage."** This panel lists any credentials already issued for your organization.
3. **Click "+ New S3 credential."** Optionally give it a label (e.g. "backup-server-prod") so you can identify it later.
4. **(Optional) Click "Restrict scope" before creating it.** You can limit a credential to one bucket, one folder path/prefix, a specific set of operations (READ / WRITE / DELETE / LIST), and/or an expiry date — useful for handing a narrowly-scoped credential to a specific backup job or third-party tool. Leave every field blank for a full-access credential.
5. **Save the Access Key ID and Secret Access Key immediately.** The secret is shown exactly once, right after creation, and can never be retrieved again — only reissued. Store it the same way you'd store any other cloud credential (a secrets manager, not a chat message or a plaintext file).

One credential, both protocols. The same Access Key ID / Secret Access Key pair works against both the S3 endpoint and the Azure endpoint — you don't need to issue a separate credential per protocol.

SECTION 03

Step 2 — Configure the AWS CLI

Set up a named profile pointed at Inaya's endpoint, exactly like you would for any other S3-compatible provider.

TERMINAL

```
aws configure --profile inaya
# AWS Access Key ID:     <your Access Key ID>
# AWS Secret Access Key: <your Secret Access Key>
# Default region name:   inaya
# Default output format: json
```

TERMINAL — EVERY COMMAND NEEDS --ENDPOINT-URL

```
aws s3 ls --profile inaya --endpoint-url https://<your-inaya-host>/api/s3
aws s3 mb s3://pilot-bucket --profile inaya --endpoint-url https://<your-inaya-host>/api/s3
aws s3 cp report.pdf s3://pilot-bucket/ --profile inaya --endpoint-url https://<your-inaya-host>/api/s3
aws s3 sync ./backups s3://pilot-bucket/backups/ --profile inaya --endpoint-url https://<your-inaya-host>/api/s3
```

Tip: export `AWS_ENDPOINT_URL=https://<your-inaya-host>/api/s3` in your shell profile so you don't need `--endpoint-url` on every command.

SECTION 04

Step 3 — Configure Azure Tooling

The Access Key ID doubles as your Azure "account name"; the Secret Access Key doubles as the account key. Point the Azure SDK/CLI at Inaya's custom Blob endpoint the same way you would any non-default Azure endpoint.

NODE.JS (@AZURE/STORAGE-BLOB)

```
const { BlobServiceClient, StorageSharedKeyCredential } = require("@azure/storage-blob");

const credential = new StorageSharedKeyCredential(
  "<your Access Key ID>",
  "<your Secret Access Key>"
);

const client = new BlobServiceClient(
  "https://<your-inaya-host>/api/azure",
  credential
);

const container = client.getContainerClient("pilot-container");
await container.createIfNotExists();
await container.getBlockBlobClient("report.pdf").uploadFile("./report.pdf");
```

Signing in with your company Microsoft account instead. Inaya also accepts a real Microsoft Entra ID (Azure AD) access token in place of the account-key credential above — useful if you'd rather authenticate as a specific signed-in user than manage a shared key. Ask your Inaya administrator to confirm this is enabled for your organization.

SECTION 05

Step 4 — Verify the Pilot Connection

1. **Create a test bucket/container.** `aws s3 mb s3://pilot-test` (or the Azure container equivalent).
2. **Upload a small test file.** Confirm it appears in a subsequent list command.
3. **Download it back and compare.** The downloaded file should be byte-for-byte identical to what you uploaded.
4. **Confirm it's visible in Business Workspace.** The object should appear under the matching bucket in the S3-Compatible Storage panel — this confirms Inaya's own storage/encryption/redundancy pipeline received it, not just a local acknowledgment.
5. **Delete the test object and bucket.** Confirms delete permissions work end-to-end before moving real data.

SECTION 06

Large Files & Bulk Transfers

- Multipart upload is fully supported for the AWS CLI/SDK — `aws s3 cp`/`aws s3 sync` automatically use it for larger files, no extra configuration needed.
- Azure's block-upload workflow (Put Block / Put Block List) is fully supported for the Azure SDK's own automatic chunking on large blobs.
- Byte-range reads (partial file downloads) are supported on both protocols — useful for resumable downloads and streaming.
- Per-part/per-block size is capped at 8MB, matching Inaya's existing internal shard size elsewhere in the platform.

SECTION 07

What's Different From a Standard Cloud Provider

Security model, stated plainly. Objects written through this compatibility layer are encrypted with a key Inaya's server manages on your organization's behalf — necessary because standard S3/Azure client tools were never built to encrypt data client-side before sending it. This is the same trust model every real S3-compatible storage gateway operates under. The key is itself encrypted at rest, scoped per organization, and every use is logged to your organization's own audit trail — visible to your admins, not just asserted. Ask your Inaya contact for the full technical security writeup if your compliance team needs it.

- Presigned URLs: AWS SigV4 presigned URLs (aws s3 presign, SDK presigners) are supported and were verified against the real AWS CLI on 27 September 2026 (expiry up to 7 days, bound to one method and one object). Google-style X-Goog-* signed URLs use the same code path but are not verified against a Google client.
- Not every third-party backup tool has been tested against this endpoint yet; ask your Inaya contact for the current tested-tool list before relying on one for production backups.

SECTION 08

Getting Help

During the pilot, route any connection issues, unexpected errors, or questions about scope/permissions to your assigned Inaya technical contact rather than general support — pilot feedback is what shapes the general-availability release.